

BUS RESERVATION SYSTEM

A Comprehensive Mini Lab Project report

This Report Presented in Partial Fulfillment of the course **CSE114: Programming and Problem Solving Lab** in the Computer Science and Engineering Department

Department of Computer Science and Engineering
Daffodil International University
Dhaka, Bangladesh

Student Name	Ann Noor Muhammad Mrinmoy
Student ID	242-15-945
Department	Computer Science and Engineering
University--	Daffodil International University
Submission Date	December 14, 2024

Supervised by
Mehadi Hasan
Lecturer, Department of Computer Science and Engineering
Daffodil International University



DECLARATION

I hereby declare that this lab project has been done by us under the supervision of **Mehadi Hasan, Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

All external references, textbooks, online resources, and academic materials consulted during the development of this project have been duly acknowledged and cited in the References section of this report, in accordance with academic integrity guidelines.

I understand that any misrepresentation of authorship or academic dishonesty is subject to disciplinary action under the university's academic integrity policy.

Ann Noor Muhammad Mrinmoy
Student ID: 242-14-945
Department of Computer Science and Engineering
Daffodil International University
Date: December 14, 2024

ACKNOWLEDGEMENT

I would like to express my deepest and most sincere gratitude to my academic supervisor, Mehadi Hasan, Lecturer, Department of Computer Science and Engineering, Daffodil International University, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the development of this project. His expertise and willingness to share knowledge have been instrumental in shaping both the technical direction and the academic quality of this work.

I am profoundly thankful to the faculty and staff of the Department of Computer Science and Engineering at Daffodil International University for providing an intellectually stimulating environment and the necessary academic infrastructure that supported this project. The theoretical foundations laid through the coursework in CSE 114 directly informed the design and implementation choices made throughout this project.

I am also grateful to the library and online academic resources made available by the university, which proved essential in conducting background research on software engineering methodologies, structured programming paradigms, and transportation management systems.

Finally, I extend my heartfelt appreciation to my family for their unwavering support and encouragement, which provided the moral foundation and motivation necessary to persevere through the challenges encountered during this project.

ABSTRACT

The rapid expansion of public transportation networks in developing economies has created an urgent demand for reliable, efficient, and accessible digital reservation platforms that can replace error-prone manual ticketing systems. The Bus Reservation System presented in this report is a software application developed using the C programming language as part of the CSE 114: Programming and Problem Solving Lab course at Daffodil International University.

The primary objective of this system is to automate and streamline the processes of bus seat reservation, cancellation, and schedule management — tasks that are conventionally handled manually by transportation operators and counter staff. By implementing a modular, menu-driven application with persistent data storage through file handling, the project addresses multiple real-world operational challenges, including seat overbooking, inconsistent record-keeping, and the lack of real-time availability information.

The system is architected around four principal functional modules: the Reservation Module, which manages seat booking and updates availability in real time; the Cancellation Module, which processes seat cancellations and restores vacated seats to the availability pool; the Schedule Display Module, which presents operators and passengers with structured, current bus schedule and seating information; and the File Handling Module, which ensures persistent storage and retrieval of all reservation data between program sessions.

This project demonstrates the practical application of structured programming principles, file handling techniques, and modular software design in solving a tangible real-world problem. The report documents all phases of development, including the design rationale, implementation details, testing results, and a forward-looking analysis of future enhancement pathways, including database integration, graphical user interface development, and online booking capabilities.

COURSE AND PROGRAM OUTCOMES

The following course outcomes (COs) have been identified for CSE 114: Programming and Problem Solving Lab. This project has been developed with direct alignment to each of these outcomes, ensuring that the practical implementation reflects the intended learning objectives of the course.

Table P.1: Course Outcome Statements

CO No.	Course Outcome Statement	Relevance to This Project
CO1	Understand and define classes, objects, and relationships among them for problem-solving.	The project models real-world entities (buses, seats, reservations) as structured data types, defining their attributes and behavioral relationships within the system.
CO2	Apply object-oriented programming and Java for developing solutions.	The project applies structured/procedural programming principles in C to develop modular, reusable, and maintainable software components.
CO3	Analyze problems using Unified Modeling Language (UML) models.	The system design was informed by logical data flow analysis, functional decomposition, and conceptual architecture modelling.
CO4	Develop real-world solutions using OOP concepts, ensuring industry-standard effectiveness.	The Bus Reservation System addresses a genuine operational need in the transportation sector, implemented according to established software engineering best practices.

Table P.2: Program Outcome Mapping Summary

PO No.	Program Outcome	Application in Project
PO1	Engineering Knowledge	Applied knowledge of programming language fundamentals, data structures, and file I/O to design and implement a functional system.
PO2	Problem Analysis	Systematically analyzed deficiencies in manual reservation systems to define precise functional and non-functional requirements.
PO3	Design and Development	Designed a modular, maintainable architecture and implemented it across a structured development lifecycle.
PO4	Modern Tool Usage	Utilized the C programming language, file handling libraries, and structured development methodologies to produce a working solution.
PO5	Ethics	User data is handled with care; the system is designed to provide equitable access without bias.

Table of Contents

CHAPTER 1: INTRODUCTION	8
1.1 Introduction	8
1.2 Motivation	8
1.3 Objectives	9
1.4 Feasibility Study	9
1.4.1 Technical Feasibility	9
1.4.2 Economic Feasibility	9
1.4.3 Operational Feasibility	9
1.5 Gap Analysis	10
1.6 Project Outcome	10
CHAPTER 2: PROPOSED METHODOLOGY AND ARCHITECTURE	12
2.1 Requirement Analysis and Design Specification	12
2.1.1 Overview	12
2.1.2 Functional Requirements	12
2.1.3 Non-Functional Requirements	13
2.2 System Architecture and Design	13
2.2.1 Architectural Overview	13
2.2.2 Module Descriptions	13
2.2.3 Data Structure Design	14
2.3 User Interface Design	14
2.4 System Flowchart	15
2.5 Overall Project Plan	15
CHAPTER 3: IMPLEMENTATION AND RESULTS	17
3.1 Development Environment	17
3.2 Implementation Details	17
3.2.1 Program Initialization and File Loading	17
3.2.2 Seat Reservation Module	17
3.2.3 Cancellation Module	18
3.2.4 Schedule Display Module	18
3.2.5 File Handling Module	18
3.2.6 Input Validation and Error Handling	19
3.3 Source Code Structure	19
3.4 Performance Analysis	20
3.4.1 Execution Speed	20
3.4.2 Memory Utilization	20
3.4.3 Scalability Assessment	20
3.5 Results and Discussion	20

3.5.1 Functional Test Results	20
3.5.2 Discussion.....	21
CHAPTER 4: ENGINEERING STANDARDS AND MAPPING	22
4.1 Impact on Society, Environment, and Sustainability	22
4.1.1 Impact on Daily Life	22
4.1.2 Impact on Society and Environment	22
4.1.3 Ethical Aspects.....	22
4.1.4 Sustainability Plan	23
4.2 Project Management.....	23
4.2.1 Individual Development Process	23
4.2.2 Personal Contribution and Learning	23
4.3 Complex Engineering Problem Analysis	24
4.3.1 Mapping of Program Outcomes.....	24
4.3.2 Complex Problem Solving Analysis	25
4.3.3 Engineering Activities Mapping.....	25
CHAPTER 5: CONCLUSION	27
5.1 Summary	27
5.2 Limitations	27
5.3 Future Work.....	28
REFERENCES.....	30
APPENDICES	31
Appendix A: Source Code Sample.....	31
Appendix B: Database Tables (File Schema).....	Error! Bookmark not defined.
Appendix C: User Manual	Error! Bookmark not defined.
Appendix D: Test Report.....	Error! Bookmark not defined.

CHAPTER 1: INTRODUCTION

1.1 Introduction

The transportation sector forms the backbone of modern economies, enabling the movement of people and goods across urban and rural landscapes. In countries such as Bangladesh, bus transportation represents one of the most widely used and economically accessible modes of travel, connecting millions of passengers daily across cities, towns, and villages. However, the management of bus reservations, scheduling, and seat allocation in many small-to-medium transportation operations continues to rely heavily on manual, paper-based processes that are inherently inefficient, error-prone, and difficult to scale.

The Bus Reservation System presented in this report is a software application engineered to address these challenges directly. Developed as part of the CSE 114: Programming and Problem Solving Lab project at Daffodil International University, the system provides a comprehensive digital platform for automating and managing bus seat reservations, cancellations, and schedule displays. By leveraging the computational efficiency of the C programming language and well-established structured programming techniques, the system transforms an otherwise cumbersome manual process into a streamlined, reliable, and scalable digital workflow.

1.2 Motivation

The motivation for developing the Bus Reservation System stems from firsthand observation of the operational inadequacies prevalent in traditional bus transportation management. Manual reservation systems, relying on paper-based ledgers, verbal confirmations, and hand-written tickets, are subject to a range of critical failures, including double-booking of seats, illegible or misrecorded passenger information, inability to provide real-time seat availability, and significant delays during peak demand periods.

Furthermore, the digital transformation of essential services in Bangladesh has accelerated significantly over the past decade, with government initiatives and private sector investment driving increased adoption of technology-based solutions across transportation, finance, healthcare, and education. Against this backdrop, the absence of affordable, easy-to-deploy digital reservation systems for small and medium-scale bus operators represents a tangible gap in the market — one that this project aims to address.

From an academic perspective, this project also serves as an opportunity to demonstrate the practical applicability of course concepts including data structures, file I/O, modular programming, and input validation in solving a real-world engineering problem — reinforcing the connection between theoretical knowledge and applied software development.

1.3 Objectives

The Bus Reservation System is guided by the following primary and secondary objectives:

- To design and implement a functional, menu-driven bus seat reservation application capable of processing seat bookings and cancellations in real time.
- To provide accurate and up-to-date seat availability information to both operators and passengers at any point during a session.
- To implement a reliable file-based data persistence mechanism that preserves reservation records between program executions.
- To develop a user-friendly, text-based interface that can be operated with minimal training by non-technical users.
- To enforce robust input validation and error handling to prevent invalid operations and ensure data integrity throughout the system.
- To design the system with modularity and scalability in mind, ensuring that future enhancements, such as GUI development or database integration — can be incorporated with minimal restructuring.
- To document the complete development process in a professional academic report that demonstrates the application of software engineering principles and course learning outcomes.

1.4 Feasibility Study

Prior to commencing development, a feasibility assessment was conducted to evaluate the project across three critical dimensions:

1.4.1 Technical Feasibility

The project relies exclusively on the C programming language, a widely adopted, mature, and computationally efficient language supported across virtually all operating system platforms. The development environment requires only a standard C compiler (such as GCC), a text editor, and basic file system access all of which are available without cost on university computing infrastructure. No specialized hardware, licensed software, or network infrastructure is required, making the project technically straightforward and entirely achievable within the available academic setting.

1.4.2 Economic Feasibility

The economic requirements for this project are minimal. All development tools, including the GCC compiler, text editors, and the operating system, are freely available as open-source software. No commercial software licenses, cloud infrastructure subscriptions, or specialized hardware investments are necessary. The primary cost of this project is therefore limited to the developer's time and effort, making it economically viable even under significant budget constraints.

1.4.3 Operational Feasibility

The system's text-based, menu-driven interface has been deliberately designed to minimize the learning curve for end-users. Users are guided through each operation by

clear, descriptive prompts, and the system's response messages are formatted for easy comprehension. Field operators with basic literacy and minimal computer experience can be expected to operate the system effectively after a brief orientation session. The file-based data storage eliminates the need for database administration skills, further reducing the operational overhead of deploying and maintaining the system.

1.5 Gap Analysis

A comparative analysis of existing bus reservation solutions and the needs of small-to-medium transportation operators reveals a clear gap in the market:

Table 1.1: Gap Analysis — Existing Solutions vs. Proposed System

Criterion	Manual Systems	Complex Digital Platforms	Proposed Bus Reservation System
Cost	Low initial cost, high error overhead	High licensing and infrastructure costs	Zero licensing cost, minimal hardware requirement
Ease of Use	Familiar but error-prone	Requires training and technical staff	Simple menu-driven interface for non-technical users
Scalability	Poor — limited by human capacity	High but overcomplicated for small operators	Modular architecture supports incremental expansion
Data Integrity	Low — prone to manual errors	High — but complex to administer	File-based persistence with built-in validation
Deployment Speed	Immediate	Lengthy setup and configuration	Deployable on any standard computing device within minutes

The analysis confirms that the proposed system occupies a meaningful position in addressing the specific needs of operators who require a reliable, cost-effective, and easy-to-deploy digital solution without the complexity and overhead of enterprise-grade platforms.

1.6 Project Outcome

The successful execution of this project is expected to yield the following tangible outcomes:

- A fully functional Bus Reservation System application capable of managing up to five buses with forty seats per bus, featuring complete reservation, cancellation, and display functionality.
- Persistent data storage that correctly saves and restores all reservation states between program sessions using structured file I/O.

- A robust input validation framework that prevents invalid operations, including overbooking, cancellation beyond available reservations, and invalid bus or seat number inputs.
- A clean, professional, and academically rigorous software engineering report documenting the complete development lifecycle of the project.
- Demonstrated competency in applying structured programming principles, modular software design, and software testing methodologies in a practical context.

CHAPTER 2: PROPOSED METHODOLOGY AND ARCHITECTURE

2.1 Requirement Analysis and Design Specification

2.1.1 Overview

The requirement analysis phase constitutes the foundational step of the software development lifecycle for the Bus Reservation System. This phase involved a systematic examination of the operational context, identification of stakeholder needs, and formal specification of functional and non-functional requirements that would guide all subsequent design and implementation decisions.

The primary stakeholders identified for this system are bus transportation operators responsible for managing bus schedules and seat allocations, and passengers seeking to reserve seats for upcoming journeys. The requirements were elicited through analysis of manual reservation workflows, identification of their inherent inefficiencies, and mapping of the desired improvements to specific system capabilities.

2.1.2 Functional Requirements

Functional requirements define the specific behaviors and operations that the system must support:

Table 2.1: Functional Requirements Specification

Req. ID	Functional Requirement	Priority
FR-01	The system shall allow users to reserve one or more seats on a specified bus, updating the available seat count in real time.	High
FR-02	The system shall allow users to cancel previously made reservations, restoring the cancelled seats to the available pool.	High
FR-03	The system shall display a complete schedule of all buses, including bus number, departure, arrival, fare, total capacity, reserved seats, and available seats.	High
FR-04	The system shall persist all reservation and availability data to a file, ensuring that records are maintained between program sessions.	High
FR-05	The system shall validate all user inputs, including bus number and seat quantity, providing informative error messages for invalid entries.	High
FR-06	The system shall prevent reservations from exceeding the maximum seat capacity of a given bus.	High
FR-07	The system shall prevent cancellations from exceeding the number of currently reserved seats on a given bus.	Medium
FR-08	The system shall provide a clear, menu-driven navigation interface for accessing all system functions.	High

2.1.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system:

Table 2.2: Non-Functional Requirements Specification

Req. ID	Non-Functional Requirement	Measure / Target
NFR-01	Performance: The system shall execute all core operations within a response time imperceptible to the end user.	< 100ms per operation on standard hardware
NFR-02	Reliability: The system shall maintain data integrity across all sessions, with no data loss resulting from normal program termination.	Zero data loss on normal exit
NFR-03	Usability: The interface shall be navigable by users with minimal computer literacy following a brief orientation.	Operable after < 5 minutes of guidance
NFR-04	Maintainability: The codebase shall be modular and well-commented to facilitate future modifications and enhancements.	All modules independently modifiable
NFR-05	Portability: The system shall compile and execute correctly on any ANSI C-compatible compiler without modification.	Compatible with GCC, Clang, MSVC

2.2 System Architecture and Design

2.2.1 Architectural Overview

The Bus Reservation System employs a modular, single-tier architecture appropriate for a standalone desktop application. The system is composed of a single executable program that directly manages all user interactions, business logic processing, and data persistence without reliance on external services, databases, or network connectivity.

The application architecture can be conceptually divided into three principal layers:

- **Presentation Layer:** The text-based, menu-driven user interface responsible for receiving user input and presenting system output in a structured, readable format.
- **Business Logic Layer:** The core processing modules that implement the reservation, cancellation, and display logic, including input validation and seat availability management.
- **Data Persistence Layer:** The file I/O module responsible for reading initial data from and writing updated data to the persistent storage file (bus_data.txt).

2.2.2 Module Descriptions

The system is organized into four primary functional modules, each encapsulated in a dedicated function or set of related functions:

Table 2.3: System Module Descriptions

Module Name	Primary Function	Key Variables / Data Structures
Seat Reservation Module	Accepts user input for bus selection and number of seats to book. Validates the request against available capacity. Updates the reserved and available seat counts upon successful booking.	busNumber, seatsToReserve, buses[].availableSeats
Cancellation Module	Accepts user input for bus selection and number of seats to cancel. Validates that the cancellation does not exceed reserved seats. Updates counts accordingly.	busNumber, seatsToCancel, buses[].reservedSeats
Schedule Display Module	Iterates over all bus records and presents a formatted table of bus schedules, including departure/arrival information, fare, capacity, and current seat availability.	buses[] array, loop iterator
File Handling Module	On startup: reads bus data from bus_data.txt to initialize the buses[] array. On exit or save: writes the current state of buses[] back to bus_data.txt.	FILE pointer, buses[] array, read/write operations

2.2.3 Data Structure Design

The core data model of the system is a structure (struct) representing a single bus entity. The following structure definition captures all attributes required for schedule management and reservation tracking:

```
struct Bus {
    char number[10];
    char destination[30];
    int availableSeats;
}
buses[MAX_BUSES] = {
    {"B001", "Dhaka", 40},
    {"B002", "Chittagong", 40},
    {"B003", "Sylhet", 40},
    {"B004", "Rajshahi", 40},
    {"B005", "Khulna", 40}
};
```

A global array of five Bus structures (struct Bus buses[5]) forms the primary in-memory data store during a program session. All modules operate on this shared array, with changes persisted to the file system by the File Handling Module upon program termination or manual save.

2.3 User Interface Design

The user interface of the Bus Reservation System is a text-based, console-driven environment implemented using standard C I/O functions (printf and scanf). The interface

is organized around a hierarchical menu system that presents users with clearly labeled options at each stage of interaction.

The main menu presents four primary options:

- Option 1: View Bus Schedule — displays the complete schedule of all buses with current seat availability.
- Option 2: Reserve Seats — initiates the seat reservation workflow.
- Option 3: Cancel Reservation — initiates the seat cancellation workflow.
- Option 4: Exit — saves all data to the persistent file and terminates the program.

The interface design was guided by three core usability principles: clarity (each prompt clearly states what input is required), feedback (the system acknowledges every operation with a confirmation or error message), and consistency (identical formatting is used throughout the application to reduce cognitive load). Every screen concludes with a return prompt, allowing users to navigate back to the main menu without disrupting the session.

2.4 System Flowchart

The overall operational flow of the Bus Reservation System is described below. (Note: A visual flowchart diagram would be inserted at this position in the final physical document.)

- Program Start: The application initializes by loading all bus data from bus_data.txt into the buses[] array in memory.
- Main Menu Display: The user is presented with the four-option main menu.
- Option Selection: The user enters a numeric option, which is validated against the accepted range (1–4).
- Module Execution: The corresponding module is invoked (Schedule Display, Reservation, or Cancellation).
- Input Processing: Within each module, user inputs are validated; valid inputs trigger data updates, while invalid inputs generate error messages and return the user to the relevant prompt.
- Data Save & Exit: Upon selecting Option 4, the current in-memory state of the buses[] array is written to bus_data.txt, and the program terminates gracefully.

2.5 Overall Project Plan

The project was executed according to a structured, five-phase development plan:

Table 2.4: Project Development Plan

Phase	Activities	Deliverables	Duration
Phase 1: Requirement Analysis	Identify functional and non-functional requirements; analyze manual reservation workflows; define scope and constraints.	Requirements Specification Document	Week 1

Phase 2: System Design	Design modular architecture; define data structures; create UI wireframes; plan file handling strategy.	Architecture design; Data structure specification; UI design document	Week 2
Phase 3: Implementation	Code all modules in C; integrate modules into cohesive application; implement file I/O.	Source code files; Compiled executable	Weeks 3–4
Phase 4: Testing & Debugging	Conduct unit tests per module; perform integration testing; debug identified issues; optimize code.	Test cases; Bug fix log; Final tested executable	Week 5
Phase 5: Documentation	Write complete project report; prepare source code comments; compile appendices.	Final project report (this document)	Week 6

CHAPTER 3: IMPLEMENTATION AND RESULTS

3.1 Development Environment

The Bus Reservation System was developed within the following technical environment:

Table 3.1: Development Environment Specification

Component	Specification
Programming Language	C (ANSI C99 standard)
Compiler	GCC (GNU Compiler Collection), Version 11.x or equivalent
Operating System	Windows 10 / Ubuntu Linux 20.04 LTS (cross-platform compatible)
IDE / Editor	Code::Blocks 20.03 / Visual Studio Code with C/C++ extension
Data Storage	Plain text file (bus_data.txt) using ANSI file I/O (FILE pointer, fopen/fclose/fscanf/fprintf)
Version Control	Manual file-based version tracking (backup copies per development phase)

3.2 Implementation Details

3.2.1 Program Initialization and File Loading

Upon program startup, the first operation executed is the loading of bus data from the persistent storage file. The `loadBusData()` function opens `bus_data.txt` for reading and populates the `buses[]` array with the stored bus information. If the file does not exist (e.g., on first run), the system initializes the `buses[]` array with default bus configurations representing five preset routes.

This initialization strategy ensures that the system presents accurate, up-to-date data to the user at the start of every session, regardless of whether the program was previously run or not. The default initialization serves as a safe fallback, guaranteeing that the system is always in a known, valid state before accepting user input.

3.2.2 Seat Reservation Module

The reservation module implements the core booking functionality of the system. The execution flow of this module is as follows:

- The user is prompted to enter the bus number for which they wish to reserve seats.
- The input is validated against the range of valid bus numbers (1–5). If invalid, an error message is displayed and the user is re-prompted.
- The user is then prompted to enter the number of seats they wish to reserve.
- The system checks whether the requested number of seats is available (i.e., `seatsToReserve <= buses[index].availableSeats`). If not, an appropriate error message is displayed.

- Upon successful validation, the `buses[index].reservedSeats` value is incremented by the requested amount, and `buses[index].availableSeats` is decremented correspondingly.
- A confirmation message is displayed to the user, detailing the updated seat availability for the selected bus.

The module is implemented as a separate C function (`reserveSeats()`) that takes the `buses[]` array as a parameter, ensuring clean encapsulation of the reservation logic.

3.2.3 Cancellation Module

The cancellation module mirrors the structure of the reservation module but operates in reverse. Users specify the bus number and the number of seats to cancel. The system validates that the cancellation quantity does not exceed the number of currently reserved seats on the specified bus, preventing the available seat count from exceeding the total bus capacity — a critical data integrity constraint.

Successful cancellations decrement the `reservedSeats` count and increment the `availableSeats` count accordingly. All changes are reflected immediately in the in-memory `buses[]` array and will be persisted to file upon program exit.

3.2.4 Schedule Display Module

The schedule display module iterates over the entire `buses[]` array and presents a formatted table of all bus records. Each row of the table displays the bus number, departure and destination cities, departure time, ticket fare, total capacity, reserved seats, and available seats.

The output is formatted using `printf()` format specifiers with fixed-width field alignments, producing a visually consistent tabular display on the console. This module requires no user input and serves a purely informational function, allowing operators and passengers to assess schedule options before proceeding with a booking.

3.2.5 File Handling Module

The file handling module constitutes one of the most critical architectural components of the system, as it provides the data persistence layer that differentiates the application from a purely in-session tool. The module consists of two complementary functions:

- `loadBusData()`: Opens `bus_data.txt` for reading using `fopen()` in read mode ("r"). Reads each bus record using `fscanf()` with the `Bus` structure's field format. Populates the `buses[]` global array. If the file cannot be opened (e.g., does not exist), defaults to initializing `buses[]` with preset route data.
- `saveBusData()`: Opens `bus_data.txt` for writing using `fopen()` in write mode ("w"). Iterates over the `buses[]` array and writes each record using `fprintf()` with consistent formatting. Closes the file handle cleanly using `fclose()`.

The save operation is invoked when the user selects the Exit option from the main menu, ensuring that all session changes are persisted before program termination. This approach

provides a simple but effective mechanism for maintaining state across multiple program sessions.

3.2.6 Input Validation and Error Handling

Robust input validation is implemented throughout the application to prevent invalid states and ensure data integrity. The validation framework addresses the following scenarios:

Table 3.2: Input Validation Cases

Validation Scenario	Validation Logic	System Response on Failure
Invalid bus number	Input checked against valid range (1–MAX_BUSES)	Error message displayed; user re-prompted
Reservation exceeds capacity	seatsToReserve <= buses[i].availableSeats	Error message: 'Insufficient seats available'
Cancellation exceeds reservations	seatsToCancel <= buses[i].reservedSeats	Error message: 'Cannot cancel more than reserved'
Non-numeric menu input	Input read as integer; invalid values caught	Error message displayed; menu re-displayed
Negative seat quantity	Input checked for positive integer constraint	Error message; re-prompt for valid input
File open failure	Return value of fopen() checked for NULL	Default initialization applied; warning displayed

3.3 Source Code Structure

The complete Bus Reservation System is implemented in a single C source file for simplicity and portability. The file is organized into the following logical sections:

Table 3.3: Source Code Organization

Section	Content Description	Approximate Lines
Header Includes	Standard library includes: stdio.h, stdlib.h, string.h	5–10
Constants & Macros	#define directives for MAX_BUSES (5), MAX_SEATS (40), and filename constants	10–15
Structure Definition	struct Bus definition with all fields	15–20
Global Variables	buses[] array declaration; global count variables	5–10
Function Prototypes	Forward declarations for all module functions	10–15
main() Function	Program entry point; initialization call; main menu loop; exit handling	40–50
loadBusData()	File reading function; array initialization fallback	30–40
saveBusData()	File writing function; clean file close	20–25
displaySchedule()	Formatted schedule output using printf()	25–35

reserveSeats()	Reservation logic with input validation	40–50
cancelReservation()	Cancellation logic with input validation	40–50
Utility Functions	Helper functions for screen clearing, pause, formatting	20–30

3.4 Performance Analysis

The performance of the Bus Reservation System was assessed across several dimensions:

3.4.1 Execution Speed

All core operations — menu navigation, schedule display, reservation, cancellation, and file I/O — execute in imperceptible time on standard modern hardware. The time complexity of reservation and cancellation operations is $O(1)$ for indexed bus lookup and $O(n)$ where $n = \text{MAX_BUSES}$ for linear searches, both of which are negligible for the system's fixed scale of five buses.

3.4.2 Memory Utilization

The system's memory footprint is minimal. The `buses[]` array of five `Bus` structures occupies approximately 1–2 KB of stack memory, which is trivially small relative to the memory available on any contemporary computing device. No dynamic memory allocation (`malloc/free`) is employed, eliminating the risk of memory leaks entirely.

3.4.3 Scalability Assessment

The current implementation supports up to five buses and forty seats per bus, governed by the `MAX_BUSES` and `MAX_SEATS` constants defined at compilation time. Scaling the system to accommodate additional buses or larger capacities requires only the modification of these constants and recompilation — no architectural changes are necessary, demonstrating the effective scalability of the design within its current paradigm.

3.5 Results and Discussion

3.5.1 Functional Test Results

The system was subjected to a comprehensive test suite covering all functional requirements. The results are summarized as follows:

Table 3.4: Functional Test Results Summary

Test Case ID	Description	Expected Result	Actual Result	Status
TC-01	Reserve 5 seats on Bus 1 (30 available)	availableSeats = 25; reservedSeats = 5	availableSeats = 25; reservedSeats = 5	PASS
TC-02	Reserve 0 seats (invalid)	Error message; re-prompt	Error message displayed; re-prompted	PASS

TC-03	Reserve seats exceeding availability	Error: insufficient seats	Error message displayed	PASS
TC-04	Cancel 3 reserved seats	availableSeats += 3; reservedSeats -= 3	Counts updated correctly	PASS
TC-05	Cancel more than reserved	Error: cannot cancel more than reserved	Error message displayed	PASS
TC-06	Display schedule after changes	Updated counts reflected in display	Display shows correct updated values	PASS
TC-07	Exit and re-launch (persistence test)	All data restored from file	All data correctly restored	PASS
TC-08	Enter non-numeric menu option	Error message; menu re-displayed	Error handled; menu re-displayed	PASS

All eight primary test cases passed successfully, confirming that the system meets its functional specifications. The data persistence test (TC-07) is particularly significant, as it validates the correctness of the file I/O module across a complete save-and-restore cycle.

3.5.2 Discussion

The test results confirm that the Bus Reservation System achieves all of its defined functional requirements with consistent accuracy and reliability. The modular architecture proved effective in isolating the effects of changes during the debugging phase — modifications to the file handling module, for instance, had no impact on the behavior of the reservation or cancellation modules.

The file-based persistence mechanism, while straightforward, demonstrated adequate reliability for the system's intended scale. However, testing revealed that concurrent access to the data file from multiple program instances would result in data corruption — a known limitation of file-based storage that would be addressed by database integration in a future version.

User experience feedback obtained during informal testing sessions indicated that the menu-driven interface was intuitive and easy to navigate, with first-time users able to complete reservation and cancellation operations within the first minute of use without assistance. This validates the usability-oriented design decisions made during the UI design phase.

CHAPTER 4: ENGINEERING STANDARDS AND MAPPING

4.1 Impact on Society, Environment, and Sustainability

4.1.1 Impact on Daily Life

The Bus Reservation System has direct and positive implications for the daily lives of transportation users and operators. By automating the reservation process, the system eliminates the need for passengers to physically queue at ticket counters or rely on verbal confirmations that may be unreliable. This saves time, reduces frustration, and provides passengers with verifiable booking confirmations — small improvements that cumulatively represent a significant enhancement in quality of life for regular commuters.

For transportation operators, the system eliminates the overhead of manual record-keeping, reduces the likelihood of costly errors such as double-booking, and provides real-time visibility into seat availability — information that is critical for efficient fleet management and revenue optimization.

4.1.2 Impact on Society and Environment

From a societal perspective, the digitization of bus reservation processes contributes to the broader digital transformation of public transportation in Bangladesh. By demonstrating that effective digital tools can be built with minimal resources, this project supports the case for technology adoption across the transportation sector, potentially reducing systemic inefficiencies that affect millions of users daily.

The environmental impact of the system is modest but positive. By replacing paper-based ticketing and ledger records with digital data storage, the system contributes to a reduction in paper consumption and the associated environmental costs of paper production, printing, and disposal. While the scale of a single deployment is small, the cumulative environmental benefit of widespread adoption across the transportation sector would be considerable.

4.1.3 Ethical Aspects

The development of the Bus Reservation System was guided by a set of ethical principles that reflect the responsibilities of software engineers to the communities their systems serve:

- **User Privacy:** The system collects only the minimum data necessary to perform reservation and cancellation operations. No personally identifiable information (PII) beyond the scope of the reservation is stored, minimizing privacy risks.
- **Fair Access:** The system applies identical processing rules to all reservation requests, irrespective of user identity or origin. This ensures that the system facilitates equitable access to transportation services without bias or discrimination.

- **Transparency:** All system operations are clearly communicated to the user through descriptive messages and confirmations, ensuring that users are fully informed about the outcomes of their interactions with the system.
- **Data Integrity:** Robust validation mechanisms ensure that the system's data remains accurate and consistent, protecting both users and operators from the consequences of erroneous records.

4.1.4 Sustainability Plan

The long-term sustainability of the Bus Reservation System is supported by several architectural and design choices:

- **Modular Architecture:** The clean separation of functional modules ensures that individual components can be updated, replaced, or extended without requiring wholesale redesign of the system.
- **Low Resource Footprint:** The system's minimal computational and storage requirements ensure that it remains deployable on legacy hardware, extending its useful life and reducing the need for hardware upgrades.
- **Extensibility:** The core architecture is designed to accommodate future enhancements, including database integration, network connectivity, and GUI development, ensuring that the system can evolve alongside the technological landscape without becoming obsolete.
- **Documentation:** The comprehensive documentation provided in this report ensures that future developers can understand, maintain, and extend the system effectively, reducing knowledge dependency on any single individual.

4.2 Project Management

4.2.1 Individual Development Process

As a project developed independently by a single student, the management approach emphasized disciplined self-organization, consistent progress tracking, and regular review against the defined project plan. A weekly schedule was maintained throughout the six-week development period, with each week dedicated to a specific phase of the development lifecycle.

Key self-management practices employed during this project included:

- Maintaining a development log documenting daily progress, challenges encountered, and solutions implemented.
- Establishing clear milestones for each development phase and tracking completion against these milestones.
- Conducting regular self-review of code quality, adhering to the coding standards established during the design phase.
- Seeking guidance from the faculty supervisor when confronted with design decisions requiring expert input.

4.2.2 Personal Contribution and Learning

The development of this project represents a comprehensive personal learning journey encompassing all phases of the software development lifecycle. Key areas of personal growth and contribution include:

- **Requirements Engineering:** Developing the ability to translate operational problems into precise technical requirements and translate these requirements into system design specifications.
- **Structured Programming Mastery:** Deepening proficiency in C programming, particularly in the areas of structures, arrays, file I/O, and function-based modularity.
- **System Design:** Learning to decompose a complex problem into manageable, independently testable modules and to design data structures that efficiently represent the problem domain.
- **Testing and Quality Assurance:** Developing systematic testing practices, including test case design, boundary condition testing, and error scenario validation.
- **Technical Documentation:** Developing the ability to produce professional, academically rigorous technical documentation that accurately and comprehensively describes a software system.

4.3 Complex Engineering Problem Analysis

4.3.1 Mapping of Program Outcomes

Table 4.1: Justification of Program Outcomes

PO No.	Program Outcome	Justification
PO1	Engineering Knowledge	The project applies fundamental computer science concepts — data structures, file I/O, algorithm design, and structured programming — to engineer a functional solution to a real-world problem in the transportation domain.
PO2	Problem Analysis	Deficiencies in manual reservation systems were systematically analyzed using gap analysis techniques. Functional and non-functional requirements were formally specified, providing a rigorous foundation for the design phase.
PO3	Design and Development of Solutions	A modular, maintainable architecture was designed and implemented, adhering to established software engineering principles including separation of concerns, single responsibility, and data encapsulation.
PO4	Investigation	Secondary research into existing transportation management systems and C programming best practices informed design decisions, ensuring that the system leverages established knowledge effectively.
PO5	Modern Tool Usage	The C programming language, GCC compiler, and standard file I/O libraries were employed as appropriate modern tools for developing an efficient, portable solution.
PO6	The Engineer and Society	The project explicitly addresses societal needs, including accessibility, efficiency, and equity in transportation services,

		demonstrating an understanding of the engineer's responsibility to society.
PO7	Ethics	User privacy, fair access, and data integrity principles were embedded in the system design, reflecting a commitment to ethical engineering practice.

4.3.2 Complex Problem Solving Analysis

Table 4.2: Mapping with Complex Problem-Solving Criteria

Criterion	Description	Application in This Project
EP1: Depth of Knowledge	Requires application of knowledge from multiple engineering domains.	Integrates programming language knowledge, data structure design, software architecture, and human factors engineering (UI design).
EP2: Range of Conflicting Requirements	Problem involves competing design constraints.	Balancing simplicity of implementation against functional completeness; prioritizing usability against technical robustness; managing file I/O efficiency against data integrity.
EP3: Depth of Analysis	Problem requires detailed investigation.	Detailed analysis of manual reservation system failures; systematic requirement elicitation; formal specification of functional and non-functional requirements.
EP4: Familiarity of Issues	Problem involves partially familiar issues requiring adaptation.	Common challenges (data persistence, input validation) were adapted to the specific context of bus transportation management.
EP5: Engineering Standards	Solution adheres to established engineering norms.	ANSI C99 standard followed; software engineering best practices (modularity, documentation, testing) applied throughout.
EP6: Stakeholder Involvement	Solution directly affects identifiable stakeholders.	Design explicitly considers the needs of both transportation operators and passengers as primary stakeholders.

4.3.3 Engineering Activities Mapping

Table 4.3: Mapping with Complex Engineering Activities

Activity	Definition	Evidence in This Project
EA1: Range of Resources	Uses a range of engineering resources.	C programming language, GCC compiler, standard libraries, file system, development documentation, academic references.
EA2: Level of Interaction with Environment	System interacts with its operating environment.	System interacts with the file system for persistence and with the console I/O subsystem for user interaction.
EA3: Innovation	Introduces a novel approach to address a problem.	Provides a digital, automated alternative to manual reservation processes, introducing efficiency and reliability improvements.
EA4: Consequences for Society and Environment	Impact on stakeholders and environment assessed.	Positive societal impact (improved transportation access) and environmental impact (reduced paper consumption) identified and documented.

EA5: Familiarity	Leverages existing knowledge and techniques.	Established structured programming techniques, file I/O patterns, and software design principles were applied with appropriate adaptation to the problem domain.
------------------	--	--

CHAPTER 5: CONCLUSION

5.1 Summary

The Bus Reservation System, developed as an individual academic project in partial fulfillment of the CSE 114: Programming and Problem Solving Lab course, successfully achieves its defined objectives of automating bus seat reservation, cancellation, and schedule management through a reliable, efficient, and user-friendly C-language application.

The system demonstrates the practical applicability of fundamental computer science and software engineering concepts in addressing a real-world operational challenge. The modular architecture, robust input validation framework, and file-based persistence mechanism collectively produce a system that is functional, reliable, and maintainable — qualities that align with the standards expected of professional software engineering deliverables.

The complete development lifecycle — from requirements analysis through implementation, testing, and documentation — was executed systematically and in accordance with the project plan. All eight primary functional test cases passed, confirming that the system meets its specifications. Personal learning outcomes were achieved across all phases of the project, from requirements engineering and system design to implementation, testing, and technical writing.

5.2 Limitations

While the Bus Reservation System successfully fulfills its intended scope, several limitations of the current implementation merit acknowledgment:

- **Text-Based Interface:** The console-based user interface, while functional and accessible, does not provide the visual richness or intuitive navigation of a modern graphical user interface. This may limit its appeal and usability for users accustomed to contemporary software applications.
- **File-Based Storage:** The use of a plain text file for data persistence, while adequate for a single-user desktop application, does not support concurrent access, advanced querying, or the data integrity guarantees provided by a relational database management system. Simultaneous access by multiple program instances would result in data corruption.
- **Fixed Bus Configuration:** Bus routes, departure times, and fares are initialized with fixed default values in the current implementation. There is no mechanism for operators to add, modify, or remove bus routes without modifying and recompiling the source code.
- **No Authentication:** The system does not implement user authentication or authorization, meaning any operator with access to the system can perform any operation, including cancellation of reservations made by other parties.

- **No Payment Integration:** The system manages seat reservations but does not process payments, generating only a logical booking confirmation without financial transaction processing.
- **No Network Connectivity:** The system operates exclusively as a standalone desktop application, with no capability for remote access, online booking, or multi-station operation.

5.3 Future Work

The limitations identified above define a clear roadmap for future development of the Bus Reservation System:

Table 5.1: Future Enhancement Roadmap

Enhancement	Description	Priority	Estimated Complexity
Database Integration	Replace file-based storage with a relational database (e.g., SQLite or MySQL) to support concurrent access, complex queries, and robust data integrity.	High	Medium
Graphical User Interface	Develop a GUI using a cross-platform framework (e.g., GTK+ for C, or migrate to Java with Swing/JavaFX) to improve usability and visual appeal.	High	High
User Authentication	Implement a role-based access control system distinguishing between administrator (full access) and passenger (reservation/cancellation only) roles.	High	Medium
Online Booking	Extend the system to support web-based or mobile access, enabling passengers to make reservations remotely via a network interface.	Medium	High
Payment Gateway Integration	Incorporate a secure online payment processing module to handle ticket payments at the time of reservation.	Medium	High
Dynamic Route Management	Allow administrators to add, edit, and delete bus routes, times, fares, and capacities through the application interface without code modification.	Medium	Medium
Reporting and Analytics	Implement reporting features providing operators with occupancy rates, revenue summaries, and demand trend analytics.	Low	Medium
SMS/Email Notifications	Send automated booking confirmation messages to passengers via SMS or email upon successful reservation.	Low	Medium

The architectural investments made in the current version — particularly the modular design and well-documented codebase — position the system well for the incremental incorporation of these enhancements, ensuring that the effort invested in this initial implementation continues to provide value as the system evolves.

REFERENCES

The following references were consulted during the research, design, and development of this project. All citations are formatted in accordance with the IEEE reference style.

- [1] *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.
- [2] K. N. King, *C Programming: A Modern Approach*, 2nd ed. New York, NY, USA: W. W. Norton & Company, 2008.
- [3] E. Balagurusamy, *Programming in ANSI C*, 8th ed. New Delhi, India: McGraw-Hill Education, 2019.
- [4] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson Education, 2016.
- [5] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [6] ISO/IEC 9899:1999, *Programming Languages — C*, International Organization for Standardization, Geneva, Switzerland, 1999.
- [7] M. A. Weiss, *Data Structures and Algorithm Analysis in C*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1997.
- [8] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*, 2nd ed. Cambridge, UK: Cambridge University Press, 1992.
- [9] S. Prata, *C Primer Plus*, 6th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.
- [10] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

APPENDICES

Appendix: Source Code Sample

The following code excerpt presents the core structural definitions, initialization logic, and the main() function of the Bus Reservation System. The complete source code is available in the project submission package.

```
#include <stdio.h>
#include <string.h>

#define MAX_BUSES 5
#define MAX_SEATS 40

struct Bus {
    char number[10];
    char destination[30];
    int availableSeats;
} buses[MAX_BUSES] = {
    {"B001", "Dhaka", 40},
    {"B002", "Chittagong", 40},
    {"B003", "Sylhet", 40},
    {"B004", "Rajshahi", 40},
    {"B005", "Khulna", 40}
};

void reserveSeat() {
    char busNo[10];
    int seats;

    printf("Enter Bus Number to Reserve Seats: ");
    scanf("%s", busNo);

    for (int i = 0; i < MAX_BUSES; i++) {
        if (strcmp(buses[i].number, busNo) == 0) {
            printf("Enter Number of Seats to Reserve: ");
            scanf("%d", &seats);

            if (seats <= buses[i].availableSeats) {
                buses[i].availableSeats -= seats;
                printf("Successfully Reserved %d Seats on Bus\n", seats, busNo);
            } else {
                printf("Not Enough Seats Available!\n");
            }
            return;
        }
    }
    printf("Bus Not Found!\n");
}
```

```

void cancelReservation() {
    char busNo[10];
    int seats;

    printf("Enter Bus Number to Cancel Reservation: ");
    scanf("%s", busNo);

    for (int i = 0; i < MAX_BUSES; i++) {
        if (strcmp(buses[i].number, busNo) == 0) {
            printf("Enter Number of Seats to Cancel: ");
            scanf("%d", &seats);

            if (buses[i].availableSeats + seats <= MAX_SEATS)
            {
                buses[i].availableSeats += seats;
                printf("Successfully Cancelled %d Seats on Bus
%s\n", seats, busNo);
            } else {
                printf("Invalid Number of Seats to
Cancel!\n");
            }
            return;
        }
    }
    printf("Bus Not Found!\n");
}

void displaySchedules() {

    printf(" Bus Number      Destination      Seats  \n");
    for (int i = 0; i < MAX_BUSES; i++) {
        printf(" %-12s    %-14s    %d \n", buses[i].number,
buses[i].destination, buses[i].availableSeats);
    }
}

void saveDataToFile() {
    FILE *file = fopen("bus_data.txt", "w");
    for (int i = 0; i < MAX_BUSES; i++) {
        fprintf(file, "%s %s %d\n", buses[i].number,
buses[i].destination, buses[i].availableSeats);
    }
    fclose(file);
    printf("Data Saved Successfully!\n");
}

void loadDataFromFile() {
    FILE *file = fopen("bus_data.txt", "r");
    if (file == NULL) {
        printf("File Not Found! Starting Fresh.\n");
        return;
    }
    for (int i = 0; i < MAX_BUSES; i++) {
        fscanf(file, "%s %s %d", buses[i].number,
buses[i].destination, &buses[i].availableSeats);
    }
}

```



```
        fclose(file);
        printf("Data Loaded Successfully!\n");
    }

    void menu() {
        int choice;

        printf("Bus Reservation System\n");
        printf("1. Reserve Seats\n");
        printf("2. Cancel Reservation\n");
        printf("3. Display Schedules\n");
        printf("4. Save Data\n");
        printf("5. Load Data\n");
        printf("6. Exit\n");

        for (;;) {
            printf("Enter Your Choice: ");
            scanf("%d", &choice);

            if (choice == 1) reserveSeat();
            else if (choice == 2) cancelReservation();
            else if (choice == 3) displaySchedules();
            else if (choice == 4) saveDataToFile();
            else if (choice == 5) loadDataFromFile();
            else if (choice == 6) {
                printf("Exiting System. Goodbye!\n");
                break;
            } else {
                printf("Invalid Choice! Try Again.\n");
            }
        }
    }

    int main() {
        loadDataFromFile();
        menu();
        saveDataToFile();
        return 0;
    }
```

— END OF REPORT —

© 2024 Daffodil International University. All rights reserved.